

Laboratorium: Algorytmy Probabilistyczne

Metody Sztucznej Inteligencji | Politechnika Gdańska | sem. letni 2025/2026

Imię i nazwisko: Igor Józefowicz

Nr indeksu: 193257

Grupa lab.: UM1

Data: 12.05.2026

Cel laboratorium

Celem tego laboratorium jest praktyczne zapoznanie się z **Naiwnym Klasyfikatorem Bayesowskim** oraz **Sieciami Bayesowskimi** - dwoma fundamentalnymi narzędziami probabilistycznej sztucznej inteligencji.

Struktura (90 min)

Zadanie	Temat	Czas
1	Naiwny Bayes krok po kroku — filtr spamu	~30 min
2	Warianty NB i wpływ założenia niezależności	~30 min
3	Sieci Bayesowskie — struktura i wnioskowanie	~30 min

Zasady

- Odpowiedzi na pytania wpisuj w wyznaczonych komórkach Markdown.
- Komórki z kodem oznaczone `# TODO` wymagają uzupełnienia.

Konfiguracja środowiska

Uruchom poniższe komórki **raz** na początku zajęć.

```
In [1]: %pip install -q scikit-learn matplotlib numpy pandas pgmpy datasets
```

Note: you may need to restart the kernel to use updated packages.

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
numba 0.61.0 requires numpy<2.2,>=1.24, but you have numpy 2.4.6 which is incompatible.
sklearn-compat 0.1.3 requires scikit-learn<1.7,>=1.2, but you have scikit-learn 1.8.0 which is incompatible.
streamlit 1.45.1 requires pandas<3,>=1.4.0, but you have pandas 3.0.2 which is incompatible.

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMa
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11
print("Środowisko gotowe!")
```

```
c:\Users\Igor\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:
22: UserWarning: Pandas requires version '2.10.2' or newer of 'numexpr' (version
'2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Środowisko gotowe!
```

Zadanie 1 – Naiwny Bayes krok po kroku: filtr spamu (30 min)

Kontekst

Na wykładzie poznaliśmy przykład klasyfikacji e-maili jako **spam** vs **ham** z użyciem cech binarnych (obecność słów). Teraz zastosujemy tę samą logikę na prawdziwym zbiorze SMS-ów.

Twierdzenie Bayesa — przypomnienie

$$P(C|\mathbf{x}) = \frac{P(\mathbf{x}|C) \cdot P(C)}{P(\mathbf{x})}$$

Naiwne założenie niezależności:

$$P(\mathbf{x}|C) = \prod_{i=1}^n P(x_i|C)$$

Wygładzanie Laplace'a (aby uniknąć zerowych prawdopodobieństw):

$$P(x_i|C) = \frac{N_{ic} + \alpha}{N_c + \alpha k}$$

Krok 1: Ręczne obliczenia - mini-przykład z wykładu

Zanim przejdziemy do prawdziwych danych, powtórzmy ręczne obliczenia z wykładu.

Mamy 6 e-maili treningowych z trzema cechami binarnymi: **oferta**, **gratulacje**, **spotkanie**.

E-mail	oferta	gratulacje	spotkanie	Klasa
1	1	1	0	spam
2	1	0	0	spam
3	0	1	0	spam
4	0	0	1	ham
5	0	0	1	ham
6	1	0	1	ham

Nowy e-mail: oferta = 1, gratulacje = 1, spotkanie = 0

Zadanie: Oblicz ręcznie, korzystając z wygładzania Laplace'a ($\alpha=1$, $k=2$), do jakiej klasy zostanie przypisany nowy e-mail. Uzupełnij poniższy kod.

```
In [3]: # === Ręczne obliczenia Naiwnego Bayesa z wygładzaniem Laplace'a ===
# Parametry:  $\alpha = 1$ ,  $k = 2$  (cechy binarne: 0 lub 1)
alpha = 1
k = 2

# Prior
P_spam = 3 / 6
P_ham = 3 / 6

# Nowy e-mail: oferta=1, gratulacje=1, spotkanie=0

# === Likelihood dla SPAM ( $N_c = 3$ ) ===
N_spam = 3
# Ile razy "oferta"=1 w klasie spam? -> e-maile 1,2 -> 2 razy
# TODO: Oblicz  $P(\text{oferta}=1 \mid \text{spam})$  z wygładzaniem Laplace'a
# Wskazówka: wzór wygładzania Laplace'a – licz wystąpienia cechy w klasie,
#          dodaj  $\alpha$  w liczniku i  $\alpha*k$  w mianowniku ( $k$  = liczba wartości)
P_of1_spam = (2 + alpha) / (N_spam + alpha * k)

# TODO: Oblicz  $P(\text{gratulacje}=1 \mid \text{spam})$ 
# Ile razy "gratulacje"=1 w klasie spam? -> e-maile 1,3 -> 2 razy
P_gr1_spam = (2 + alpha) / (N_spam + alpha * k)

# TODO: Oblicz  $P(\text{spotkanie}=0 \mid \text{spam})$ 
# Ile razy "spotkanie"=0 w klasie spam? -> e-maile 1,2,3 -> 3 razy
P_sp0_spam = (3 + alpha) / (N_spam + alpha * k)

# === Likelihood dla HAM ( $N_c = 3$ ) ===
N_ham = 3
```

```

# TODO: Oblicz P(oferta=1 | ham) – e-mail 6 -> 1 raz
P_of1_ham = (1 + alpha) / (N_ham + alpha * k)

# TODO: Oblicz P(gratulacje=1 | ham) – żaden e-mail ham -> 0 razy
P_gr1_ham = (0 + alpha) / (N_ham + alpha * k)

# TODO: Oblicz P(spotkanie=0 | ham)
# Uwaga: e-maile 4,5 mają spotkanie=1, e-mail 6 też ma spotkanie=1, więc spotkan
P_sp0_ham = (0 + alpha) / (N_ham + alpha * k)

# === Posterior (nienormalizowany) ===
posterior_spam = P_spam * P_of1_spam * P_gr1_spam * P_sp0_spam
posterior_ham = P_ham * P_of1_ham * P_gr1_ham * P_sp0_ham

print(f"P(spam|x) ∝ {P_spam} × {P_of1_spam:.3f} × {P_gr1_spam:.3f} × {P_sp0_spam:.3f}")
print(f"P(ham|x) ∝ {P_ham} × {P_of1_ham:.3f} × {P_gr1_ham:.3f} × {P_sp0_ham:.3f}")
print(f"\nDecyzja MAP: {'SPAM' if posterior_spam > posterior_ham else 'HAM'}")

# Normalizacja do prawdopodobieństw
total = posterior_spam + posterior_ham
print(f"\nP(spam|x) = {posterior_spam/total:.3f}")
print(f"P(ham|x) = {posterior_ham/total:.3f}")

```

$P(\text{spam}|x) \propto 0.5 \times 0.600 \times 0.600 \times 0.800 = 0.1440$
 $P(\text{ham}|x) \propto 0.5 \times 0.400 \times 0.200 \times 0.200 = 0.0080$

Decyzja MAP: SPAM

$P(\text{spam}|x) = 0.947$
 $P(\text{ham}|x) = 0.053$

Krok 2: Weryfikacja za pomocą sklearn

Sprawdźmy, czy nasze obliczenia są zgodne z implementacją `BernoulliNB` z biblioteki `sklearn`.

```

In [4]: # Weryfikacja za pomocą sklearn BernoulliNB
from sklearn.naive_bayes import BernoulliNB

# Dane treningowe z tabeli
X_train_mini = np.array([
    [1, 1, 0], # spam
    [1, 0, 0], # spam
    [0, 1, 0], # spam
    [0, 0, 1], # ham
    [0, 0, 1], # ham
    [1, 0, 1], # ham
])
y_train_mini = np.array([1, 1, 1, 0, 0, 0]) # 1=spam, 0=ham

# Nowy e-mail
X_new = np.array([[1, 1, 0]])

bnb = BernoulliNB(alpha=1.0)
bnb.fit(X_train_mini, y_train_mini)

pred = bnb.predict(X_new)
proba = bnb.predict_proba(X_new)

```

```
print(f"Predykcja sklearn: {'SPAM' if pred[0] == 1 else 'HAM'}")
print(f"P(spam|x) = {proba[0][1]:.3f}")
print(f"P(ham|x) = {proba[0][0]:.3f}")
print(f"\nCzy wynik zgodny z ręcznymi obliczeniami? Porównaj wartości powyżej.")
```

Predykcja sklearn: SPAM
 $P(\text{spam}|x) = 0.947$
 $P(\text{ham}|x) = 0.053$

Czy wynik zgodny z ręcznymi obliczeniami? Porównaj wartości powyżej.

Tak, wartości policzone ręcznie są identyczne z wynikami z BernoulliNB z sklearn

Krok 3: Prawdziwe dane - SMS Spam Collection

Teraz zastosujemy Naiwnego Bayesa do prawdziwego zbioru danych SMS Spam z HuggingFace. Zbiór zawiera ~5500 wiadomości SMS oznaczonych jako `ham` lub `spam`.

```
In [7]: from datasets import load_dataset
import pandas as pd

dataset = load_dataset("ucirvine/sms_spam", "plain_text", split="train")
df = pd.DataFrame(dataset)

print(f"Rozmiar zbioru: {len(df)}")
print("\nRozkład klas:")
print(df["label"].value_counts().rename({0: "ham", 1: "spam"}))

print("\nPrzykładowe wiadomości:")
for label_name, label_val in [("HAM", 0), ("SPAM", 1)]:
    sample = df[df["label"] == label_val]["sms"].iloc[0]
    print(f" [{label_name}] {sample[:100]}...")

README.md: 0%|          | 0.00/4.98k [00:00<?, ?B/s]
plain_text/train-00000-of-00001.parquet: 0%|          | 0.00/359k [00:00<?, ?B/s]
Generating train split: 0%|          | 0/5574 [00:00<?, ? examples/s]
Rozmiar zbioru: 5574

Rozkład klas:
label
ham      4827
spam      747
Name: count, dtype: int64

Przykładowe wiadomości:
[HAM] Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got a...
[SPAM] Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entr...
```

```
In [8]: # Przygotowanie danych
X_text = df['sms'].tolist()          # Lista stringów (Arrow → Python List)
y = df['label'].to_numpy(dtype=int)  # Arrow → numpy array

X_train_txt, X_test_txt, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Wektoryzacja tekstu – CountVectorizer zamienia tekst na macierz zliczeń słów
vectorizer = CountVectorizer(max_features=3000, stop_words='english')
X_train_counts = vectorizer.fit_transform(X_train_txt)
X_test_counts = vectorizer.transform(X_test_txt)

print(f"Macierz cech (train): {X_train_counts.shape}")
print(f"Macierz cech (test): {X_test_counts.shape}")
print(f"Przykładowe cechy (słowa): {vectorizer.get_feature_names_out()[:10]}")
print(f"Przykładowe cechy (słowa): {vectorizer.get_feature_names_out()[-10:]}")
```

Macierz cech (train): (4459, 3000)

Macierz cech (test): (1115, 3000)

Przykładowe cechy (słowa): ['00' '000' '02' '0207' '03' '04' '05' '0578' '06' '07821230901']

Przykładowe cechy (słowa): ['yr' 'yrs' 'yummy' 'yun' 'yunny' 'yuo' 'yup' 'zed' 'zindgi' 'zoe']

Krok 4: MultinomialNB na danych tekstowych

MultinomialNB jest wariantem idealnym do danych tekstowych (zliczenia słów).

Wytrenuj model i oceń jego jakość.

Zadanie: Uzupełnij kod, wytrenuj model i przeanalizuj macierz pomyłek.

```
In [9]: # Twój kod – MultinomialNB
# Dokumentacja: sklearn.naive_bayes – klasa MultinomialNB
# Wskazówka: model sklearn ma metody .fit() i .predict() (patrz BernoulliNB w Kr

# TODO: Utwórz model MultinomialNB z wygładzaniem Laplace'a (parametr alpha)
mnb = MultinomialNB(alpha=1.0)

# TODO: Wytrenuj model na danych treningowych (macierz cech + etykiety)
# TODO: mnb.???(???, ???)
mnb.fit(X_train_counts, y_train)

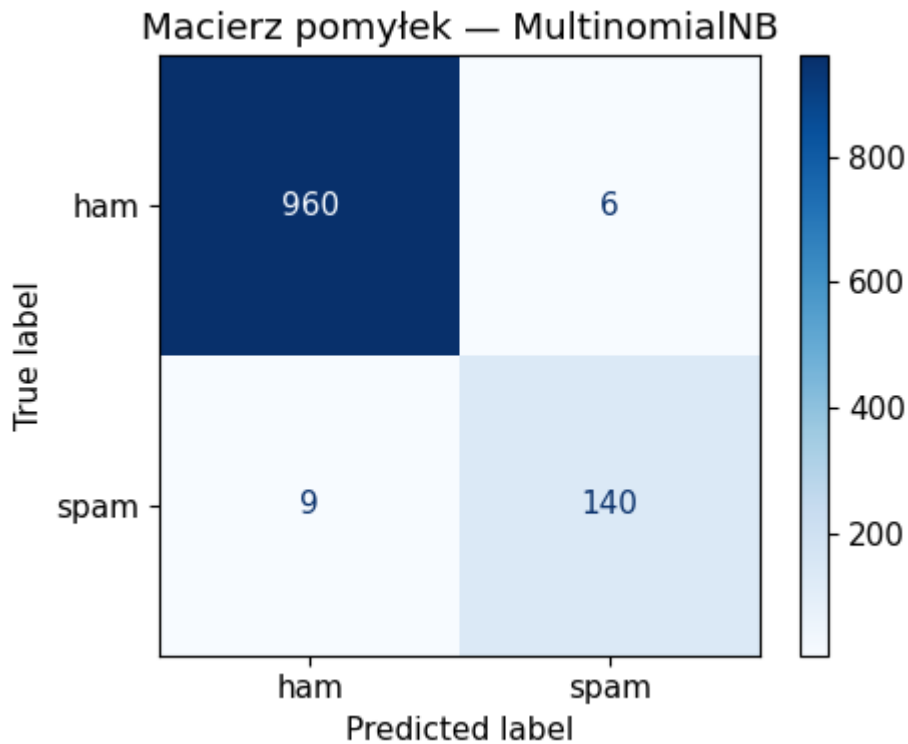
# TODO: Oblicz predykcje na zbiorze testowym
y_pred = mnb.predict(X_test_counts)

# Wyświetl raport klasyfikacji
print("Raport klasyfikacji (MultinomialNB):")
print(classification_report(y_test, y_pred, target_names=['ham', 'spam']))

# Macierz pomyłek
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, display_labels=['ham', 'spam'])
ax.set_title('Macierz pomyłek – MultinomialNB')
plt.tight_layout()
plt.show()
```

Raport klasyfikacji (MultinomialNB):

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	966
spam	0.96	0.94	0.95	149
accuracy			0.99	1115
macro avg	0.97	0.97	0.97	1115
weighted avg	0.99	0.99	0.99	1115



Krok 5: Wpływ wygładzania Laplace'a

Zbadajmy, jak parametr α wpływa na jakość klasyfikacji.

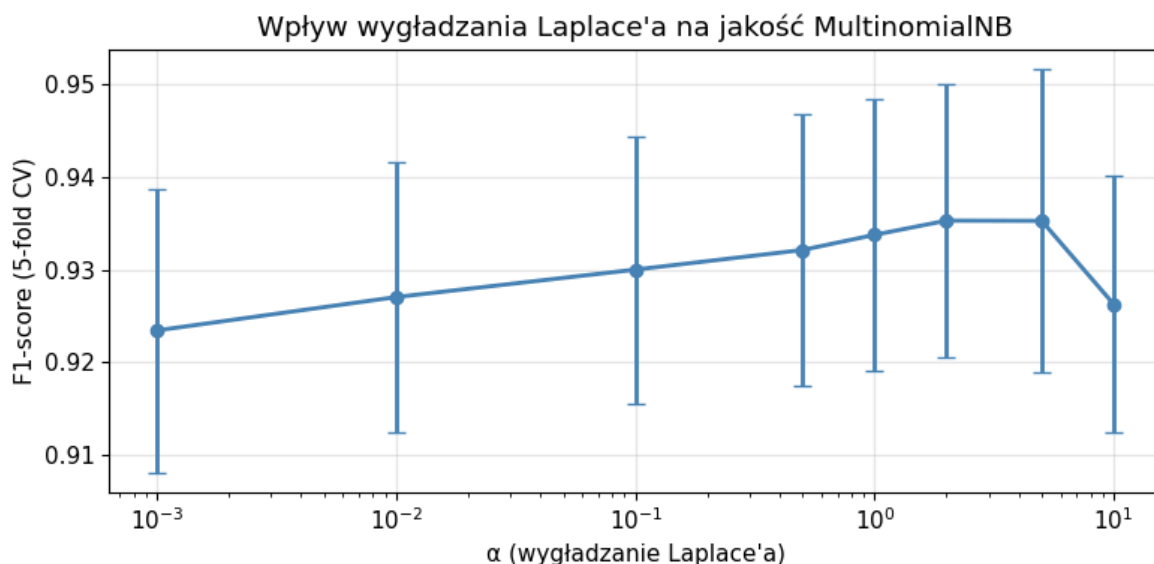
```
In [10]: # Eksperyment: wpływ parametru alpha
alphas = [0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
results = []

for a in alphas:
    mnb_a = MultinomialNB(alpha=a)
    scores = cross_val_score(mnb_a, X_train_counts, y_train, cv=5, scoring='f1')
    results.append({'alpha': a, 'f1_mean': scores.mean(), 'f1_std': scores.std()})

df_alpha = pd.DataFrame(results)

fig, ax = plt.subplots(figsize=(8, 4))
ax.errorbar(df_alpha['alpha'], df_alpha['f1_mean'], yerr=df_alpha['f1_std'],
            marker='o', capsize=4, linewidth=2, color='steelblue')
ax.set_xscale('log')
ax.set_xlabel('α (wygładzanie Laplace\'a)')
ax.set_ylabel('F1-score (5-fold CV)')
ax.set_title('Wpływ wygładzania Laplace\'a na jakość MultinomialNB')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

best = df_alpha.loc[df_alpha['f1_mean'].idxmax()]
print(f"Najlepsze α = {best['alpha']} → F1 = {best['f1_mean']:.4f}")
```



Najlepsze $\alpha = 2.0 \rightarrow F1 = 0.9353$

```
In [11]: # === Ręczne obliczenia Naiwnego Bayesa z wygładzaniem Laplace'a ===
# ALPHA USTAWIONE NA 0
alpha = 0
k = 2

# Prior
P_spam = 3 / 6
P_ham = 3 / 6

# Nowy e-mail: oferta=1, gratulacje=1, spotkanie=0

# === Likelihood dla SPAM (N_c = 3) ===
N_spam = 3
# Ile razy "oferta"=1 w klasie spam? -> e-maile 1,2 -> 2 razy
# TODO: Oblicz P(oferta=1 | spam) z wygładzaniem Laplace'a
# Wskazówka: wzór wygładzania Laplace'a – licz wystąpienia cechy w klasie,
#          dodaj alpha w liczniku i alpha*k w mianowniku (k = liczba wartości)
P_of1_spam = (2 + alpha) / (N_spam + alpha * k)

# TODO: Oblicz P(gratulacje=1 | spam)
# Ile razy "gratulacje"=1 w klasie spam? -> e-maile 1,3 -> 2 razy
P_gr1_spam = (2 + alpha) / (N_spam + alpha * k)

# TODO: Oblicz P(spotkanie=0 | spam)
# Ile razy "spotkanie"=0 w klasie spam? -> e-maile 1,2,3 -> 3 razy
P_sp0_spam = (3 + alpha) / (N_spam + alpha * k)

# === Likelihood dla HAM (N_c = 3) ===
N_ham = 3
# TODO: Oblicz P(oferta=1 | ham) – e-mail 6 -> 1 raz
P_of1_ham = (1 + alpha) / (N_ham + alpha * k)

# TODO: Oblicz P(gratulacje=1 | ham) – żaden e-mail ham -> 0 razy
P_gr1_ham = (0 + alpha) / (N_ham + alpha * k)

# TODO: Oblicz P(spotkanie=0 | ham)
# Uwaga: e-maile 4,5 mają spotkanie=1, e-mail 6 też ma spotkanie=1, więc spotkan
P_sp0_ham = (0 + alpha) / (N_ham + alpha * k)

# === Posterior (nienormalizowany) ===
posterior_spam = P_spam * P_of1_spam * P_gr1_spam * P_sp0_spam
```



```

posterior_ham = P_ham * P_of1_ham * P_gr1_ham * P_sp0_ham

print(f"P(spam|x) ∝ {P_spam} × {P_of1_spam:.3f} × {P_gr1_spam:.3f} × {P_sp0_spam:.3f}")
print(f"P(ham|x) ∝ {P_ham} × {P_of1_ham:.3f} × {P_gr1_ham:.3f} × {P_sp0_ham:.3f}")
print(f"\nDecyzja MAP: {'SPAM' if posterior_spam > posterior_ham else 'HAM'}")

# Normalizacja do prawdopodobieństw
total = posterior_spam + posterior_ham
print(f"\nP(spam|x) = {posterior_spam/total:.3f}")
print(f"P(ham|x) = {posterior_ham/total:.3f}")

```

$P(\text{spam}|x) \propto 0.5 \times 0.667 \times 0.667 \times 1.000 = 0.2222$

$P(\text{ham}|x) \propto 0.5 \times 0.333 \times 0.000 \times 0.000 = 0.0000$

Decyzja MAP: SPAM

$P(\text{spam}|x) = 1.000$

$P(\text{ham}|x) = 0.000$

Pytania do Zadania 1

P1.1 W ręcznych obliczeniach (Krok 1): co by się stało, gdybyśmy **nie** zastosowali wygładzania Laplace'a ($\alpha=0$)? Która cecha w której klasie spowodowałaby problem i dlaczego?

Twoja odpowiedź:

=== Wyniki dla alfa=0 ===

$P(\text{spam}|x) = 1.000$

$P(\text{ham}|x) = 0.000$

=== Wyniki dla alfa=1 ===

$P(\text{spam}|x) \propto 0.5 \times 0.600 \times 0.600 \times 0.800 = 0.1440$

$P(\text{ham}|x) \propto 0.5 \times 0.400 \times 0.200 \times 0.200 = 0.0080$

$P(\text{spam}|x) = 0.947$

$P(\text{ham}|x) = 0.053$

=== Wnioski ===

Bez wygładzania wyszło $P(\text{spam}|x) = 1$ i $P(\text{ham}|x) = 0$

Stało się tak, ponieważ cechy "gratulacje" i "spotkanie" nie występują ani razu w pożądanym mailach (HAM), co spowodowało pomnożenie przez 0 i stąd wynik $P(\text{ham}|x)$ jest równy 0

P1.2 Spójrz na macierz pomyłek. Jaki typ błędu jest częstszy: fałszywie pozytywne (ham sklasyfikowany jako spam) czy fałszywie negatywne (spam sklasyfikowany jako ham)? Który z tych błędów jest **bardziej kosztowny** w praktyce i dlaczego?

Twoja odpowiedź:

True label spam & Predicted label ham: 9

True label ham & Predicted label spam: 6

Czyli okazało się, że więcej było przypadków false negative (spam sklasyfikowany jako ham) -> 9 przypadków.

Moim zdaniem bardziej kosztowny i bolesny dla właściciela konta pocztowego jest sytuacja gdzie nie dostaje ważnego maila do głównej skrzynki, bo trafia do skrzynki spamu i tego maila nie zauważa niż sytuacja gdzie spam trafia do głównej skrzynki i właściciel musi (zapewne w ciągu kilkudziesięciu sekund) przeczytać spam i go ręcznie usunąć / oznaczyć jako spam.

Czyli bardziej bolesne są false positive w przypadku tego zagadnienia.

P1.3 Na wykresie wpływu α : co oznacza bardzo małe α (np. 0.001)? Co oznacza bardzo duże α (np. 10.0)? Dlaczego ekstremalnie małe i duże wartości α obniżają jakość klasyfikacji?

Twoja odpowiedź:

Bardzo małe alfa, typu alfa = 0.001, oznacza bardzo słabe wygładzanie. Model mocno dopasowuje się do danych treningowych. Rzadko występujące lub niewidziane wcześniej słowa otrzymują prawdopodobieństwo bliskie zeru -> problem zerowych prawdopodobieństw, w tym przypadku bliskich zera

Bardzo duże alfa, np. alfa = 10.0 -> oznacza bardzo silne wygładzanie. Dodany parametr może mocno zaciemniać faktyczną liczbę wystąpień słów w danych i powodować underfitting.

Czyli podsumowując, małe alfa obniża jakość przez przeuczenie i zerowanie wyników -> słabo reaguje na nowe zdania, natomiast ekstremalnie duże alfa zaciemnia prawdziwe zależności językowe -> wszystkie słowa wydają się modelowi podobnie prawdopodobne w każdej klasie.

Zadanie 2 - Warianty NB i wpływ założenia niezależności (30 min)

Kontekst

Na wykładzie poznaliśmy trzy warianty Naiwnego Bayesa:

Wariant	Typ danych	Rozkład $P(x_i C)$
GaussianNB	cechy ciągłe	rozkład normalny
MultinomialNB	zliczenia, tekst	rozkład wielomianowy
BernoulliNB	cechy binarne	rozkład Bernoulliego

Dowiedzieliśmy się też, że NB jest modelem **generatywnym** (modeluje $P(C|x)$), w przeciwieństwie do **regresji logistycznej** (model dyskryminatywny, modeluje $P(C|x)$).

W tym zadaniu zbadamy te różnice na danych liczbowych.

Krok 1: Załaduj zbiór danych Iris

```
In [12]: from sklearn.datasets import load_iris, load_wine

# Iris – 4 cechy ciągłe, 3 klasy, 150 próbek
iris = load_iris()
X_iris, y_iris = iris.data, iris.target

print(f"Iris: {X_iris.shape[0]} próbek, {X_iris.shape[1]} cechy, {len(np.unique(
print(f"Nazwy cech: {iris.feature_names}")
print(f"Nazwy klas: {list(iris.target_names)}")
```

Iris: 150 próbek, 4 cechy, 3 klasy

Nazwy cech: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Nazwy klas: [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]

Krok 2: Porównanie wariantów NB na danych ciągłych

Zadanie: Porównaj GaussianNB, MultinomialNB i BernoulliNB na zbiorze Iris. Który wariant jest odpowiedni dla jakich danych?

```
In [13]: # Porównanie wariantów NB na Iris
from sklearn.preprocessing import MinMaxScaler

# MultinomialNB wymaga danych nieujemnych – skalujemy do [0, 1]
X_iris_mm = MinMaxScaler().fit_transform(X_iris)

models = {
    'GaussianNB': GaussianNB(),
    'MultinomialNB': MultinomialNB(alpha=1.0),
    'BernoulliNB': BernoulliNB(alpha=1.0),
}

print("Porównanie wariantów NB na zbiorze Iris (5-fold CV):")
print(f"{'Model':<18} {'Accuracy':>10} {'Std':>8}")
print("-" * 38)

for name, model in models.items():
    # MultinomialNB potrzebuje danych nieujemnych
    data = X_iris_mm if name == 'MultinomialNB' else X_iris
    scores = cross_val_score(model, data, y_iris, cv=5, scoring='accuracy')
    print(f"{'name':<18} {'scores.mean():>10.3f} {'scores.std():>8.3f}")
```

Porównanie wariantów NB na zbiorze Iris (5-fold CV):

Model	Accuracy	Std
GaussianNB	0.953	0.027
MultinomialNB	0.793	0.049
BernoulliNB	0.333	0.000

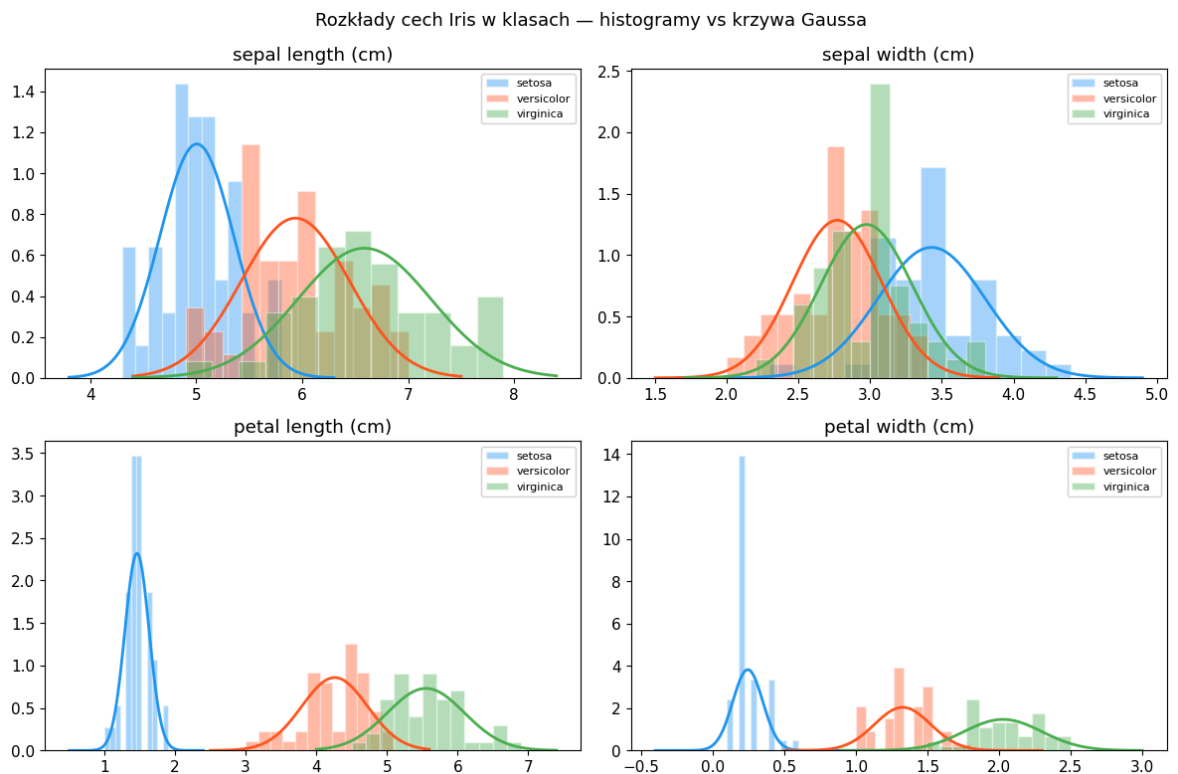
Krok 3: Wizualizacja założenia GaussianNB

GaussianNB zakłada, że cechy w każdej klasie mają rozkład normalny. Sprawdźmy, na ile to prawda dla danych Iris.

```
In [14]: # Wizualizacja rozkładów cech w klasach – czy pasują do Gaussa?
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
colors = ['#2196F3', '#FF5722', '#4CAF50']

for idx, (ax, fname) in enumerate(zip(axes.flat, iris.feature_names)):
    for cls in range(3):
        data_cls = X_iris[y_iris == cls, idx]
        ax.hist(data_cls, bins=12, alpha=0.4, color=colors[cls],
                label=iris.target_names[cls], density=True, edgecolor='white')
        # Dopasowanie Gaussa
        mu, sigma = data_cls.mean(), data_cls.std()
        x_range = np.linspace(data_cls.min() - 0.5, data_cls.max() + 0.5, 100)
        gauss = (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x_range - mu) ** 2) / sigma ** 2)
        ax.plot(x_range, gauss, color=colors[cls], linewidth=2)
    ax.set_title(fname)
    ax.legend(fontsize=8)

fig.suptitle('Rozkłady cech Iris w klasach – histogramy vs krzywa Gaussa', fontsize=12)
plt.tight_layout()
plt.show()
```



Krok 4: Badanie korelacji cech — łamanie założenia niezależności

Kluczowe założenie Naiwnego Bayesa: cechy są **warunkowo niezależne** przy danej klasie. Sprawdźmy, czy to prawda.

Zadanie: Oblicz macierz korelacji cech **w obrębie każdej klasy** i przeanalizuj, jak silne są zależności.

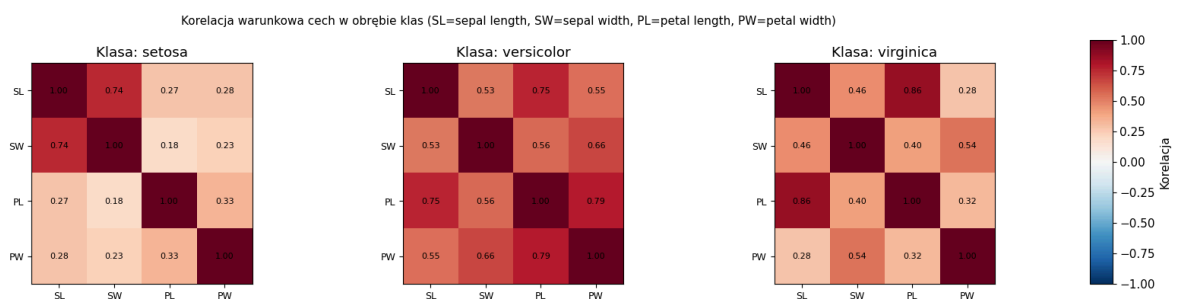
```
In [15]: # Macierze korelacji cech w obrębie klas
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for cls in range(3):
    X_cls = X_iris[y_iris == cls]
    corr = np.corrcoef(X_cls.T)
    im = axes[cls].imshow(corr, vmin=-1, vmax=1, cmap='RdBu_r')
    axes[cls].set_title(f'Klasa: {iris.target_names[cls]}')
    axes[cls].set_xticks(range(4))
    axes[cls].set_yticks(range(4))
    short_names = ['SL', 'SW', 'PL', 'PW'] # sepal/petal length/width
    axes[cls].set_xticklabels(short_names, fontsize=9)
    axes[cls].set_yticklabels(short_names, fontsize=9)
    # Anotacje
    for i in range(4):
        for j in range(4):
            axes[cls].text(j, i, f'{corr[i,j]:.2f}', ha='center', va='center', f

fig.suptitle('Korelacja warunkowa cech w obrębie klas (SL=sepal length, SW=sepal
plt.tight_layout()

# Colorbar umieszczony na zewnątrz po prawej stronie
cbar_ax = fig.add_axes([1.01, 0.1, 0.02, 0.8])
fig.colorbar(im, cax=cbar_ax, label='Korelacja')

plt.show()
```



Krok 5: NB vs Regresja Logistyczna — model generatywny vs dyskryminatywny

Na wykładzie porównywaliśmy NB (generatywny) z regresją logistyczną (dyskryminatywny). Zbadajmy to empirycznie na różnych rozmiarach zbioru treningowego.

Pytanie kluczowe: Czy NB rzeczywiście lepiej radzi sobie z małymi zbiorami danych?

```
In [19]: # NB vs Logistic Regression – krzywe uczenia
from sklearn.model_selection import learning_curve

X_iris_scaled = StandardScaler().fit_transform(X_iris)

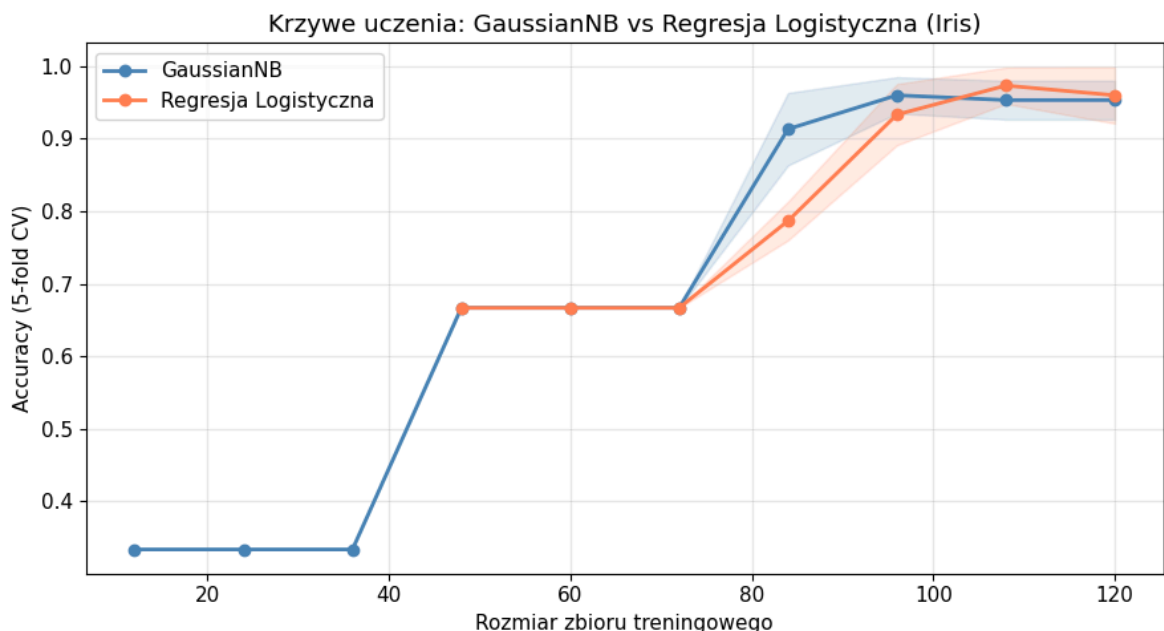
fig, ax = plt.subplots(figsize=(9, 5))

models_compare = {
    'GaussianNB': GaussianNB(),
    'Regresja Logistyczna': LogisticRegression(max_iter=1000, random_state=42),
}

train_sizes = np.linspace(0.1, 1.0, 10)
colors_lc = ['steelblue', 'coral']

for (name, model), color in zip(models_compare.items(), colors_lc):
    sizes, train_scores, test_scores = learning_curve(
        model, X_iris_scaled, y_iris, train_sizes=train_sizes,
        cv=5, scoring='accuracy', random_state=42
    )
    test_mean = test_scores.mean(axis=1)
    test_std = test_scores.std(axis=1)
    ax.plot(sizes, test_mean, 'o-', label=name, color=color, linewidth=2)
    ax.fill_between(sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)

ax.set_xlabel('Rozmiar zbioru treningowego')
ax.set_ylabel('Accuracy (5-fold CV)')
ax.set_title('Krzywe uczenia: GaussianNB vs Regresja Logistyczna (Iris)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Krok 6: Test na zbiorze z silnie skorelowanymi cechami

Stwórzmy syntetyczny zbiór danych, w którym cechy są **silnie skorelowane**, i zobaczmy jak NB sobie z tym radzi.

```
In [16]: # Syntetyczny zbiór: cechy niezależne vs skorelowane
from sklearn.datasets import make_classification

# Zbiór 1: cechy niezależne (idealne dla NB)
X_indep, y_indep = make_classification(
    n_samples=500, n_features=10, n_informative=10, n_redundant=0,
    n_clusters_per_class=1, random_state=42
)

# Zbiór 2: cechy redundantne (skorelowane) – łamie założenie NB
X_redun, y_redun = make_classification(
    n_samples=500, n_features=10, n_informative=5, n_redundant=5,
    n_clusters_per_class=1, random_state=42
)

print("Porównanie NB vs Reg. Logistyczna na danych o różnej korelacji cech:")
print(f"{'Zbiór':<25} {'GaussianNB':>12} {'Reg. Log.':>12}")
print("-" * 52)

for name, X_d, y_d in [('Cechy niezależne', X_indep, y_indep),
                       ('Cechy skorelowane', X_redun, y_redun)]:
    gnb_sc = cross_val_score(GaussianNB(), X_d, y_d, cv=5, scoring='accuracy').mean()
    lr_sc = cross_val_score(LogisticRegression(max_iter=1000), X_d, y_d, cv=5, scoring='accuracy').mean()
    print(f"{'Zbiór':<25} {'GaussianNB':>12.3f} {'Reg. Log.':>12.3f}")
```

Porównanie NB vs Reg. Logistyczna na danych o różnej korelacji cech:

Zbiór	GaussianNB	Reg. Log.
Cechy niezależne	0.892	0.880
Cechy skorelowane	0.966	0.994

Pytania do Zadania 2

P2.1 Który wariant NB dał najlepsze wyniki na Iris? Dlaczego – jakiego typu są cechy w zbiorze Iris? Dlaczego BernoulliNB radzi sobie słabo na tych danych?

Twoja odpowiedź:

Najlepsze wyniki dla zbioru Iris dał GaussianNB -> Accuracy = 0.953, bo cechy w tym zbiorze są ciągłe -> wymiar płatków itp.

Potem zauważalnie gorzej MultinomialNB -> Accuracy = 0.793

Najgorzej poradził sobie BernoulliNB -> Accuracy = 0.333, ponieważ BernoulliNB oczekuje cech binarnych, a tutaj takie nie są. Klasyfikator prawdopodobnie doprowadził te cechy do wartości 1 przez co model dla każdego gatunku widział same 1, więc nie miał jak sensownie odróżnić klas od siebie, przez co ma wynik na poziomie klasyfikatora losowego lub takiego co przewiduje tę samą klasę za każdym razem.

P2.2 Spójrz na macierze korelacji warunkowej. Czy założenie niezależności cech jest spełnione dla Iris? Wskaż konkretne pary cech, które je łamią. Mimo to GaussianNB radzi sobie dobrze — dlaczego?

Twoja odpowiedź:

Założenie niezależności cech dla zbioru Iris nie jest spełnione.

Przykładami tej sytuacji są:

W klasie virginica -> PL & SL -> korelacja = 0.86

W klasie versicolor -> PL & PW -> korelacja = 0.79

W klasie setosa -> SW & SL -> korelacja = 0.74

Wynika to z tego, że liście rosną mniej lub bardziej równo, ale nie całkowicie w oderwaniu od drugiego wymiaru liścia.

Co ciekawe, pomimo tego GaussianNB radzi sobie rzeczywiście dobrze. Pomimo, że model jest zbyt pewny procentowo, to i tak podejmuje w większości sytuacji dobrze (w tym przypadku). Dla klasyfikacji to nie ma większego znaczenia, nie robi problemu. Poza tym te klasy w zbiorze Iris są w miarę dobrze odseparowane.

P2.3 Na podstawie krzywych uczenia: przy jakim rozmiarze zbioru treningowego NB jest konkurencyjny z regresją logistyczną? Kiedy regresja logistyczna zaczyna wygrywać? Jak to się odnosi do porównania generatywne vs dyskryminatywne z wykładu?

Twoja odpowiedź:

GaussianNB staje się konkurencyjny dla regresji logistycznej przy rozmiarze zbioru treningowego ≥ 50 . Właśnie w okolicach rozmiaru 50 widać zauważalne polepszenie się dokładności i potem przechodząc do rozmiaru 80~120 dokładność też się zwiększa w podobnym tempie co regresja logistyczna.

Z wykresu wynika, że regresja logistyczna uzyskuje lepszą dokładność od rozmiaru około 100 i więcej.

Modele generatywne, takie jak NB, często są konkurencyjne przy małych danych, a modele dyskryminatywne, takie jak regresja logistyczna, zwykle wygrywają przy większej liczbie danych.

P2.4 W eksperymencie z cechami niezależnymi vs skorelowanymi: jak zmienił się stosunek wyników NB do Reg. Logistycznej? Co to mówi o praktycznym znaczeniu założenia niezależności?

Twoja odpowiedź:

Dla cech niezależnych GaussianNB uzyskał wynik 0.892, a regresja logistyczna 0.880, więc NB był minimalnie lepszy. To jest zgodne z założeniami modelu, bo naiwny Bayes najlepiej pasuje do sytuacji, w której cechy są w przybliżeniu niezależne warunkowo względem klasy.

Dla cech skorelowanych oba modele osiągnęły wysoką jakość, ale regresja logistyczna była lepsza: 0.994 wobec 0.966 dla GaussianNB. Pokazuje to, że naruszenie założenia niezależności nie musi całkowicie psuć NB, ale może osłabić jego przewagę i sprawić, że model dyskryminatywny lepiej wykorzysta zależności między cechami.

W praktyce założenie niezależności jest więc ważne, ale nie absolutne. NB często działa dobrze mimo jego naruszenia, jednak przy silnie skorelowanych cechach trzeba sprawdzić wyniki empirycznie.

Zadanie 3 – Sieci Bayesowskie: struktura i wnioskowanie (30 min)

Kontekst

Sieć Bayesowska to para (G, θ) , gdzie G to DAG (skierowany graf acykliczny), a θ to zbiór tablic CPT.

Na wykładzie poznaliśmy klasyczny przykład **sieci alarmowej**:

- **Włamanie (B)** i **Trzęsienie (E)** mogą wywołać **Alarm (A)**
- Alarm może spowodować, że **John (J)** lub **Mary (M)** zadzwonią

Użyjemy biblioteki **pgmpy** do zbudowania tej sieci i przeprowadzenia wnioskowania.

```
In [17]: # Budowa sieci alarmowej z wykładu
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Krok 1: Definiujemy strukturę grafu (krawędzie DAG)
model = DiscreteBayesianNetwork([
    ('Burglary', 'Alarm'),
    ('Earthquake', 'Alarm'),
    ('Alarm', 'JohnCalls'),
    ('Alarm', 'MaryCalls'),
])

print("Węzły:", model.nodes())
print("Krawędzie:", model.edges())
```

```
Węzły: ['Burglary', 'Alarm', 'Earthquake', 'JohnCalls', 'MaryCalls']
Krawędzie: [('Burglary', 'Alarm'), ('Alarm', 'JohnCalls'), ('Alarm', 'MaryCalls'), ('Earthquake', 'Alarm')]
```

```
In [18]: # Krok 2: Definiujemy tablice CPT (wartości z wykładu)
# Konwencja pgmpy: wartości [P(False), P(True)]

# P(Burglary): P(B=T) = 0.001
cpd_burglary = TabularCPD('Burglary', 2, [[0.999], [0.001]])

# P(Earthquake): P(E=T) = 0.002
```

```

cpd_earthquake = TabularCPD('Earthquake', 2, [[0.998], [0.002]])

# P(Alarm | Burglary, Earthquake) – tablica z wykładu
cpd_alarm = TabularCPD('Alarm', 2,
    [[0.999, 0.71, 0.06, 0.05], # P(A=False)
     [0.001, 0.29, 0.94, 0.95]], # P(A=True)
    evidence=['Burglary', 'Earthquake'],
    evidence_card=[2, 2]
)

# P(JohnCalls | Alarm)
cpd_john = TabularCPD('JohnCalls', 2,
    [[0.95, 0.10], # P(J=False)
     [0.05, 0.90]], # P(J=True)
    evidence=['Alarm'], evidence_card=[2]
)

# P(MaryCalls | Alarm)
cpd_mary = TabularCPD('MaryCalls', 2,
    [[0.99, 0.30], # P(M=False)
     [0.01, 0.70]], # P(M=True)
    evidence=['Alarm'], evidence_card=[2]
)

model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)

# Weryfikacja poprawności sieci
print(f"Sieć poprawna: {model.check_model()}")
print(f"\nCPT dla Alarm:")
print(cpd_alarm)

```

Sieć poprawna: True

CPT dla Alarm:

Burglary	Burglary(0)	Burglary(0)	Burglary(1)	Burglary(1)
Earthquake	Earthquake(0)	Earthquake(1)	Earthquake(0)	Earthquake(1)
Alarm(0)	0.999	0.71	0.06	0.05
Alarm(1)	0.001	0.29	0.94	0.95

Krok 3: Wnioskowanie — eliminacja zmiennych

Teraz przeprowadzimy różne typy wnioskowania, o których mówiliśmy na wykładzie:

- **Diagnostyczne** (bottom-up): obserwujemy skutki → wnioskujemy o przyczynach
- **Przyczynowe** (top-down): znamy przyczyny → przewidujemy skutki
- **Explaining away**: obserwowanie jednej przyczyny wpływa na wiarę w drugą

```

In [19]: # Wnioskowanie za pomocą eliminacji zmiennych
inference = VariableElimination(model)

# === Zapytanie 1: Wnioskowanie diagnostyczne ===
# "John zadzwonił – jakie jest prawdopodobieństwo włamania?"
q1 = inference.query(['Burglary'], evidence={'JohnCalls': 1})

```

```

print("=" * 50)
print("Zapytanie 1 (diagnostyczne):")
print("John zadzwonił → P(Burglary)?")
print(q1)

# === Zapytanie 2: Wnioskowanie przyczynowe ===
# "Wiemy, że jest włamanie – jaka szansa, że John zadzwoni?"
q2 = inference.query(['JohnCalls'], evidence={'Burglary': 1})
print("\n" + "=" * 50)
print("Zapytanie 2 (przyczynowe):")
print("Włamanie = True → P(JohnCalls)?")
print(q2)

```

```

=====
Zapytanie 1 (diagnostyczne):
John zadzwonił → P(Burglary)?
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(0) | 0.9837 |
+-----+-----+
| Burglary(1) | 0.0163 |
+-----+-----+

=====
Zapytanie 2 (przyczynowe):
Włamanie = True → P(JohnCalls)?
+-----+-----+
| JohnCalls | phi(JohnCalls) |
+-----+-----+
| JohnCalls(0) | 0.1510 |
+-----+-----+
| JohnCalls(1) | 0.8490 |
+-----+-----+

```

Krok 4: Explaining away — kluczowy efekt w sieciach bayesowskich

Explaining away to zjawisko, w którym obserwowanie jednej przyczyny zmniejsza wiarę w drugą przyczynę, gdy znamy wspólny skutek.

Zadanie: Uzupełnij zapytania, aby zbadać efekt explaining away.

```

In [20]: # === Explaining away ===

# Scenariusz A: Alarm się włączył – P(Burglary)?
qA = inference.query(['Burglary'], evidence={'Alarm': 1})
print("Scenariusz A: Alarm = True")
print(f"P(Burglary=True | Alarm=True) = {qA.values[1]:.4f}")

# TODO: Scenariusz B: Alarm się włączył I wiemy, że było trzęsienie – P(Burglary)
# Dokumentacja: pgmpy.inference – klasa VariableElimination, metoda .query()
# Wskazówka: analogicznie do Scenariusza A, ale evidence zawiera dwie obserwacje
qB = inference.query(['Burglary'], evidence={'Alarm': 1, 'Earthquake': 1})
print(f"\nScenariusz B: Alarm = True, Earthquake = True")
print(f"P(Burglary=True | Alarm=True, Earthquake=True) = {qB.values[1]:.4f}")

print(f"\n→ Zmiana: {qA.values[1]:.4f} → {qB.values[1]:.4f}")

```

```
print(f"→ Wiedza o trzęsieniu {'zmniejszyła' if qB.values[1] < qA.values[1] else  
      f"wierę we włamanie (explaining away)")
```

Scenariusz A: Alarm = True

$P(\text{Burglary}=\text{True} \mid \text{Alarm}=\text{True}) = 0.3736$

Scenariusz B: Alarm = True, Earthquake = True

$P(\text{Burglary}=\text{True} \mid \text{Alarm}=\text{True}, \text{Earthquake}=\text{True}) = 0.0033$

→ Zmiana: $0.3736 \rightarrow 0.0033$

→ Wiedza o trzęsieniu zmniejszyła wiarę we włamanie (explaining away)

Krok 5: Porównanie NB i ogólnej sieci bayesowskiej

Na wykładzie widzieliśmy, że Naiwny Bayes to szczególny przypadek sieci bayesowskiej (wszystkie cechy niezależne od siebie, zależne tylko od klasy). Porównajmy te dwa podejścia.

```
In [21]: # Wizualizacja: NB jako sieć bayesowska vs ogólna sieć  
# Użyjemy pgmpy do pokazania struktury  
  
# NB: klasa → każda cecha (brak krawędzi między cechami)  
nb_net = DiscreteBayesianNetwork([  
    ('Class', 'x1'),  
    ('Class', 'x2'),  
    ('Class', 'x3'),  
)  
  
# Ogólna sieć: dozwolone dodatkowe krawędzie  
general_net = DiscreteBayesianNetwork([  
    ('Class', 'x1'),  
    ('Class', 'x2'),  
    ('Class', 'x3'),  
    ('x1', 'x2'), # dodatkowa zależność  
    ('x2', 'x3'), # dodatkowa zależność  
)  
  
print("Naiwny Bayes – krawędzie:")  
for edge in nb_net.edges():  
    print(f" {edge[0]} → {edge[1]}")  
print(f" Liczba krawędzi: {len(nb_net.edges())}")  
  
print(f"\nOgólna sieć – krawędzie:")  
for edge in general_net.edges():  
    print(f" {edge[0]} → {edge[1]}")  
print(f" Liczba krawędzi: {len(general_net.edges())}")  
  
print(f"\n→ NB 'tnie' krawędzie między cechami")  
print(f"→ Ogólna sieć pozwala modelować zależności między cechami")
```

Naiwny Bayes – krawędzie:

Class → x1

Class → x2

Class → x3

Liczba krawędzi: 3

Ogólna sieć – krawędzie:

Class → x1

Class → x2

Class → x3

x1 → x2

x2 → x3

Liczba krawędzi: 5

→ NB 'tnie' krawędzie między cechami

→ Ogólna sieć pozwala modelować zależności między cechami

Krok 6: d-separacja - sprawdzanie niezależności warunkowej

d-separacja to kryterium pozwalające odczytać z grafu, które zmienne są warunkowo niezależne.

Zadanie: Sprawdź poniższe pytania o niezależność warunkową w sieci alarmowej.

```
In [23]: # d-separacja w sieci alarmowej
from pgmpy.base import DAG

print("Testy d-separacji w sieci alarmowej:")
print("=" * 60)

dag = DAG(model.edges())

tests = [
    # (zmienna1, zmienna2, obserwowane, opis)
    ('Burglary', 'Earthquake', set(), 'B ⊥ E (a priori)?'),
    ('Burglary', 'Earthquake', {'Alarm'}, 'B ⊥ E | Alarm? (explaining away)'),
    ('JohnCalls', 'MaryCalls', set(), 'John ⊥ Mary (a priori)?'),
    ('JohnCalls', 'MaryCalls', {'Alarm'}, 'John ⊥ Mary | Alarm?'),
    ('Burglary', 'JohnCalls', set(), 'B ⊥ John (a priori)?'),
    ('Burglary', 'JohnCalls', {'Alarm'}, 'B ⊥ John | Alarm?'),
]

for var1, var2, observed, desc in tests:
    # is_dconnected: True jeśli jest aktywna ścieżka (d-połączone), False jeśli
    d_connected = dag.is_dconnected(var1, var2, observed=list(observed)) if obser
    result = not d_connected # d-separated = not d-connected
    symbol = '⊥' if result else '⊥\u0338'
    print(f" {desc:<45} → {symbol} (niezależne: {result})")
```

Testy d-separacji w sieci alarmowej:

```
=====
B ⊥ E (a priori)?                → ⊥ (niezależne: True)
B ⊥ E | Alarm? (explaining away) → ⊥ (niezależne: False)
John ⊥ Mary (a priori)?          → ⊥ (niezależne: False)
John ⊥ Mary | Alarm?             → ⊥ (niezależne: True)
B ⊥ John (a priori)?             → ⊥ (niezależne: False)
B ⊥ John | Alarm?                → ⊥ (niezależne: True)
```

Pytania do Zadania 3

P3.1 W Zapytaniu 1 (diagnostyczne): John zadzwonił - jakie jest $P(\text{Burglary}=\text{True})$? Czy wartość jest wysoka czy niska? Dlaczego? Uwzględnij w odpowiedzi niskie prawdopodobieństwo a priori włamania (0.001).

Twoja odpowiedź:

Po obserwacji, że John zadzwonił, prawdopodobieństwo włamania wynosi $P(\text{Burglary}=\text{True} \mid \text{JohnCalls}=\text{True}) = 0.0163$, czyli 1.63%. Jest to dużo więcej niż prawdopodobieństwo a priori $P(\text{Burglary}=\text{True}) = 0.001$, ale nadal jest to niska wartość bezwzględna.

Wynika to z tego, że telefon Johna jest tylko pośrednim i niepewnym objawem alarmu, a samo włamanie jest bardzo rzadkie. John może zadzwonić także z powodu fałszywego alarmu albo alarmu wywołanego inną przyczyną.

P3.2 Wyjaśnij efekt *explaining away* na podstawie wyników z Kroku 4. Dlaczego wiedza o trzęsieniu ziemi zmniejsza prawdopodobieństwo włamania, gdy wiemy, że alarm się włączył? Podaj intuicyjne wyjaśnienie.

Twoja odpowiedź:

Efekt *explaining away* widać w zmianie $P(\text{Burglary}=\text{True} \mid \text{Alarm}=\text{True}) = 0.3736$ na $P(\text{Burglary}=\text{True} \mid \text{Alarm}=\text{True}, \text{Earthquake}=\text{True}) = 0.0033$.

Sam alarm mocno zwiększa szansę na włamanie, bo włamanie jest jedną z możliwych przyczyn alarmu. Gdy jednak dodatkowo wiemy, że było trzęsienie ziemi, pojawia się alternatywne wyjaśnienie alarmu. Trzęsienie "wyjaśnia" alarm, więc potrzeba zakładania włamania staje się dużo mniejsza.

P3.3 Przeanalizuj wyniki d-separacji. Dlaczego Burglary i Earthquake są niezależne a priori, ale stają się zależne po obserwacji Alarmu? Do którego z trzech typów połączeń z wykładu (serial, rozwidlenie, zbieżność) odpowiada ta sytuacja?

Twoja odpowiedź:

Burglary i Earthquake są niezależne a priori, ponieważ w modelu nie ma między nimi bezpośredniej zależności. Są to dwie oddzielne możliwe przyczyny alarmu.

Po obserwacji Alarmu stają się zależne, bo informacja o jednej przyczynie wpływa na prawdopodobieństwo drugiej. Jeśli alarm się włączył i wiemy, że było trzęsienie, to włamanie staje się mniej prawdopodobnym wyjaśnieniem.

Jest to typ połączenia zbieżnego: Burglary $\rightarrow$ Alarm $\leftarrow$ Earthquake. W takim układzie przyczyny są niezależne, dopóki wspólny skutek nie zostanie zaobserwowany.

P3.4 Dlaczego JohnCalls i MaryCalls są niezależne, gdy znamy Alarm, ale zależne, gdy Alarm jest nieobserwowany? Który typ połączenia to opisuje?

Twoja odpowiedź:

JohnCalls i MaryCalls są zależne, gdy Alarm jest nieobserwowany, ponieważ oba zdarzenia mają wspólną przyczynę: Alarm. Jeśli wiemy, że John zadzwonił, to rośnie prawdopodobieństwo, że alarm się włączył, a przez to rośnie też prawdopodobieństwo, że Mary zadzwoniła.

Gdy znamy wartość Alarmu, JohnCalls i MaryCalls stają się niezależne warunkowo. Wiedząc już, czy alarm się włączył, telefon Johna nie daje dodatkowej informacji o telefonie Mary.

To odpowiada połączeniu typu rozwidlenie: JohnCalls $\leftarrow$ Alarm $\rightarrow$ MaryCalls, gdzie obserwacja wspólnej przyczyny blokuje zależność między skutkami.

Podsumowanie — tabela porównawcza

Uzupełnij tabelę na podstawie wyników z laboratorium i wiedzy z wykładu.

Cecha	Naiwny Bayes	Ogólna Sieć Bayesowska
Założenie o cechach	Zakłada warunkową niezależność cech przy znanej klasie. Wszystkie cechy zależą od klasy, ale nie zależą bezpośrednio od siebie.	Nie wymaga pełnej niezależności cech. Zależności są opisane grafem DAG i tablicami CPT.
Złożoność uczenia	Niska, ponieważ zamiast jednej dużej tablicy zależności uczymy wiele prostych rozkładów jednowymiarowych.	Wyższa; uczenie struktury jest trudne, a tablice CPT mogą rosnąć wykładniczo wraz z liczbą rodziców węzła.
Interpretowalność	Wysoka, bo łatwo sprawdzić prior klasy i wpływ pojedynczych cech na klasyfikację.	Wysoka, jeśli struktura grafu jest sensownie dobrana; pozwala interpretować zależności przyczynowe i warunkowe.
Kiedy stosować?	Gdy potrzebny jest szybki, prosty model klasyfikacyjny, szczególnie dla tekstu, małych zbiorów danych lub wielu cech.	Gdy ważne jest modelowanie zależności między zmiennymi, wnioskowanie przy niepełnych danych i analiza przyczynowo-skutkowa.
Przykłady zastosowań	Filtrowanie spamu, klasyfikacja tekstu, analiza sentymentu,	Diagnostyka medyczna, ocena ryzyka, diagnoza awarii, wspomaganie decyzji, robotyka i

Cecha	Naiwny Bayes	Ogólna Sieć Bayesowska
	kategoryzacja dokumentów, wstępna diagnostyka.	lokalizacja, modele przyczynowości.